Perfect. I'll document **exactly what we have decided**, without changing context, so you can use this in your project documentation.

📘 AI Data Insight Dashboard with Chat Interface

Architecture & Design Documentation (Version 1)

1️⃣ Project Overview

We are building an **AI-powered Data Insight Dashboard** that allows:

- Uploading a CSV dataset
- Storing it in a distributed SQL database
- Asking natural language questions about the data
- Getting accurate numerical results
- Receiving AI-generated explanations

The system combines:

- Deterministic SQL execution
- AI-based query parsing
- AI-based explanation generation

This ensures:

- Accurate calculations
- No hallucinated math
- Scalable backend architecture
- Production-style system design

2️⃣ Core Technologies Used

Database

- YugabyteDB
  (PostgreSQL-compatible distributed SQL database)

AI Model

- Llama 3

AI Runtime

- Ollama

Backend

- Express.js (Node.js)

Frontend

- React (Vite)

3️⃣ High-Level System Architecture

```
React (Frontend)
        ↓
Express Backend API
        ↓
Active Dataset Context
        ↓
AI Query Parser (LLaMA 3 via Ollama)
        ↓
Validated Query Builder
        ↓
YugabyteDB
        ↓
Result
        ↓
AI Explanation Layer
        ↓
Frontend Response
```

4️⃣ Core Architectural Principles

We strictly follow these rules:

1. AI never performs mathematical computation.
2. Database performs all aggregations.
3. AI only translates natural language → structured query.
4. AI explains computed results.
5. Backend validates everything before execution.

This prevents:

- Hallucinated results
- Incorrect calculations
- Unsafe SQL execution

5️⃣ Dataset Handling Strategy

We Chose:

✅ One active dataset at a time
✅ New table per CSV upload
✅ Metadata tracking table

6️⃣ Database Design

6.1 Metadata Table

```
CREATE TABLE datasets (
  id UUID PRIMARY KEY,
  name TEXT,
  table_name TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  schema_json JSONB
);
```

Purpose:

- Track uploaded datasets
- Store schema
- Map dataset ID → actual table name

6.2 Dynamic Dataset Tables

For every CSV upload:

A new table is created:

```
dataset_<uuid>
```

Example:

```sql
CREATE TABLE dataset_f83a9c (
  age INT,
  income FLOAT,
  default BOOLEAN
);
```

Each dataset gets its own relational table.

Reason:

- Clean SQL aggregation
- Better performance
- Easier AI → SQL generation
- Scalable analytics design

7️⃣ CSV Upload Flow

When user uploads CSV:

1. Backend receives file
2. CSV is parsed
3. Column types inferred
4. UUID generated
5. Dynamic table created
6. Rows inserted
7. Metadata stored in `datasets`
8. Dataset marked as active (in memory)

No raw data is sent to AI.

8️⃣ Query Processing Flow (Core Feature)

When user asks a question:

Example:

> "What is the average income?"

Step 1 — AI Query Parsing

We send to LLaMA 3:

- User question
- Available column names
- Allowed operations

AI returns structured JSON:

```
{
  "operation": "mean",
  "column": "income"
}
```

AI acts as:
Natural Language → Structured Query Translator

Step 2 — Backend Validation

Backend validates:

- Operation is allowed
- Column exists
- No malicious structure

AI output is never trusted directly.

Step 3 — SQL Generation

Backend converts JSON into SQL:

```
SELECT AVG(income) FROM dataset_f83a9c;
```

Step 4 — Database Execution

YugabyteDB executes aggregation.

Database performs:

- AVG
- SUM
- COUNT
- GROUP BY
- MIN / MAX
- CORRELATION (if implemented)

Step 5 — AI Explanation Layer

After getting result:

Example result:

```
52000
```

We send to AI:

User asked:
"What is the average income?"

Computed result:
52000

AI generates:

- Interpretation
- Business reasoning
- Insight

AI does explanation only.

9️⃣ Supported Operations (Version 1 Scope)

Allowed operations:

- mean
- sum
- min
- max
- count
- group_by
- correlation (optional advanced)
- filter (basic WHERE clause)

All operations must be whitelisted.

🔟 Active Dataset Strategy

We decided:

✓ Only one dataset active at a time
✓ Active dataset stored in backend memory
✓ Frontend works only with current dataset

This simplifies:

- Query execution
- AI prompt construction
- System complexity

1️⃣1️⃣ Security & Validation Strategy

To prevent unsafe behavior:

1. Whitelist operations
2. Validate column names
3. Prevent raw SQL from AI
4. Never execute AI-generated SQL directly
5. Construct SQL programmatically

AI output is treated as suggestion, not authority.

1️⃣2️⃣ Why This Architecture Is Strong

This design demonstrates:

- Understanding of LLM limitations
- Separation of deterministic and generative systems
- Proper SQL-backed analytics
- Distributed database usage
- Safe AI integration
- Production-style architecture

This is not a chatbot.
This is a hybrid AI + data intelligence system.

1️⃣3️⃣ System Responsibility Breakdown

React

- File upload
- Display charts
- Chat interface
- Show AI insights

Express Backend

- CSV parsing
- Table creation
- Data insertion
- Query orchestration
- AI communication

YugabyteDB

- Data storage
- Aggregation
- SQL execution

LLaMA 3 (via Ollama)

- Natural language query parsing
- Insight explanation

1️⃣ 4️⃣ Future Scalability

This architecture allows future upgrades:

- Multi-dataset switching
- User authentication
- Persistent chat history
- Saved dashboards
- Cloud deployment
- Horizontal scaling (native to YugabyteDB)

No major migration required later.

✅ Final Confirmed Decisions

- Use YugabyteDB (Postgres-compatible)
- Create new table per CSV upload
- Maintain metadata table
- Allow one active dataset at a time
- Use AI-powered query parser
- Database performs calculations
- AI performs reasoning and explanation
- Strict validation of AI output
- Modular backend structure

If you want, I can next generate:

- 📄 A professional README.md version
- 📊 A system design diagram description
- 🧱 Backend folder structure documentation
- 🧠 AI prompt templates documentation

Tell me what format you want this documented in.